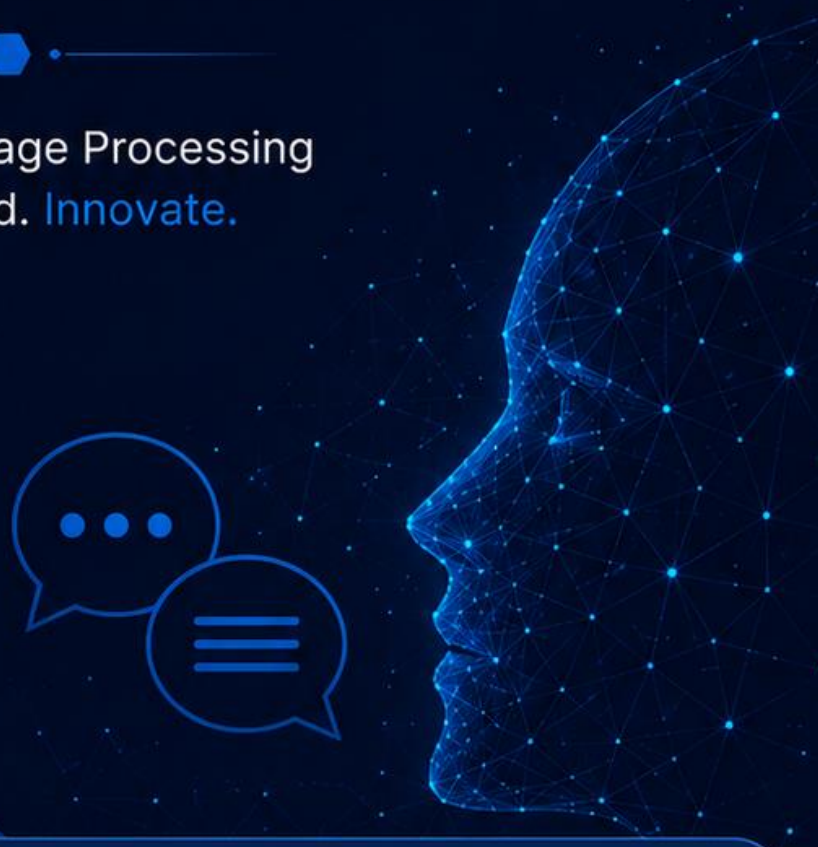
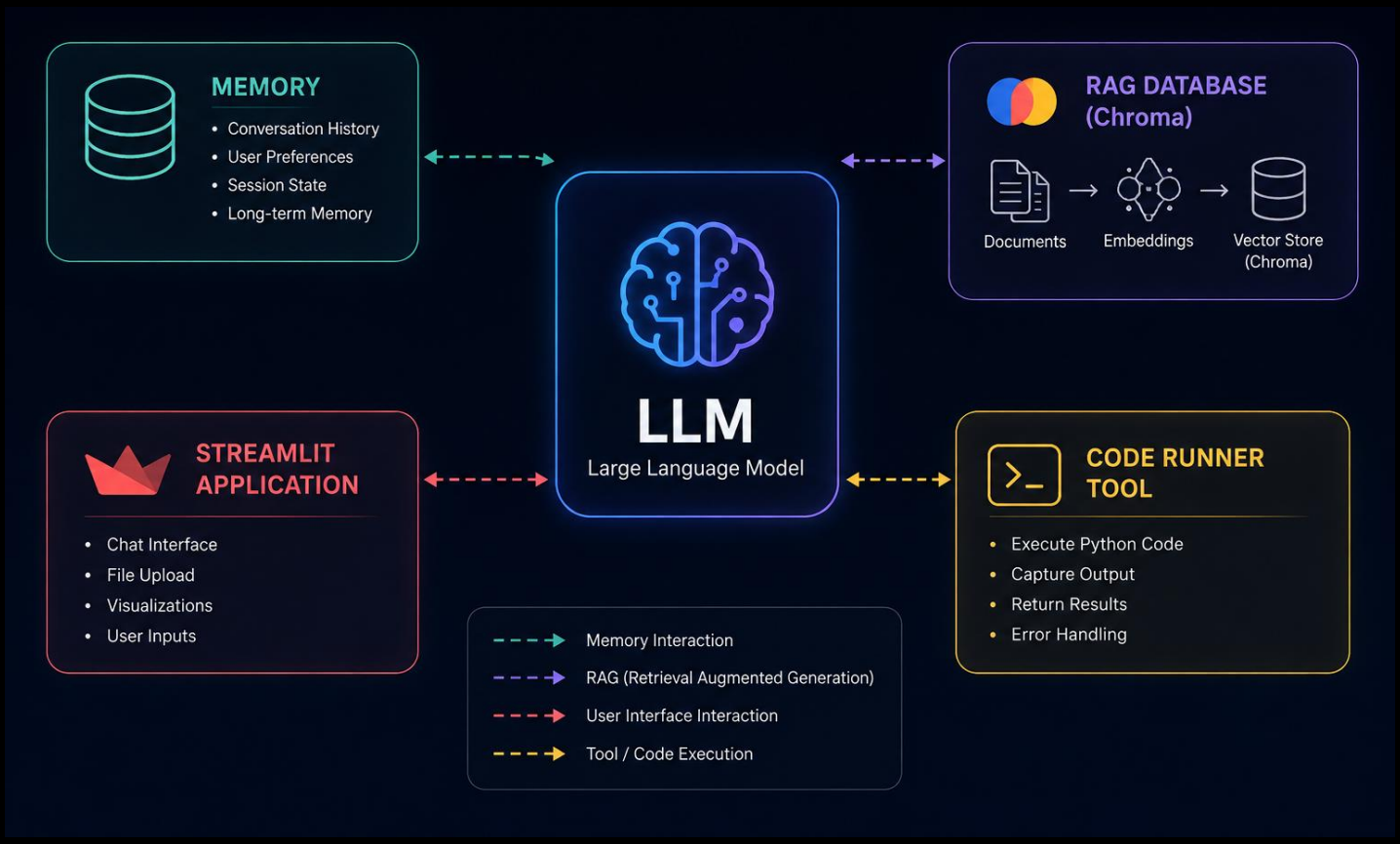
Cellula Technologies

NLP INTERNSHIP

Natural Language Processing
Learn. Build. Innovate.



Project 2


MEMORY
Conversation History
User Preferences
Session State
Long-term Memory
RAG DATABASE (Chroma)
Documents
Embeddings
Vector Store (Chroma)
LLM
Large Language Model
STREAMLIT APPLICATION
Chat Interface
File Upload
Visualizations
User Inputs
CODE RUNNER TOOL
Execute Python Code
Capture Output
Return Results
Error Handling
Memory Interaction
RAG (Retrieval Augmented Generation)
User Interface Interaction
Tool / Code Execution

Project Overview

Develop an **Intelligent AI Coding Assistant** capable of answering programming-related questions, generating code, explaining existing code, maintaining conversation memory, retrieving knowledge from a **Chroma Vector Database (VDB)** using **Retrieval-Augmented Generation (RAG)**, executing generated code through a dedicated tool, and providing an interactive user interface using **Streamlit**.

The assistant should not simply answer every request using the same workflow. Instead, it must intelligently classify the user's intent, route the request through the appropriate processing pipeline, evaluate the quality of retrieved knowledge, and continuously improve its knowledge base through user feedback.

Functional Requirements

1. User Interface

Develop a Streamlit application that provides:

- Chat interface
- Conversation history
- File upload (optional)
- Response streaming
- Code blocks with syntax highlighting
- Button to execute generated code
- Memory visualization (optional)

2. User Query Classification

Every incoming user query must first pass through an **LLM-based Intent Classifier**.

The classifier should categorize each request into one of two classes:

A. Code Explanation

Examples:

- Explain this code.
- What does this function do?

- Why is this loop incorrect?
- Explain line by line.

B. Code Generation

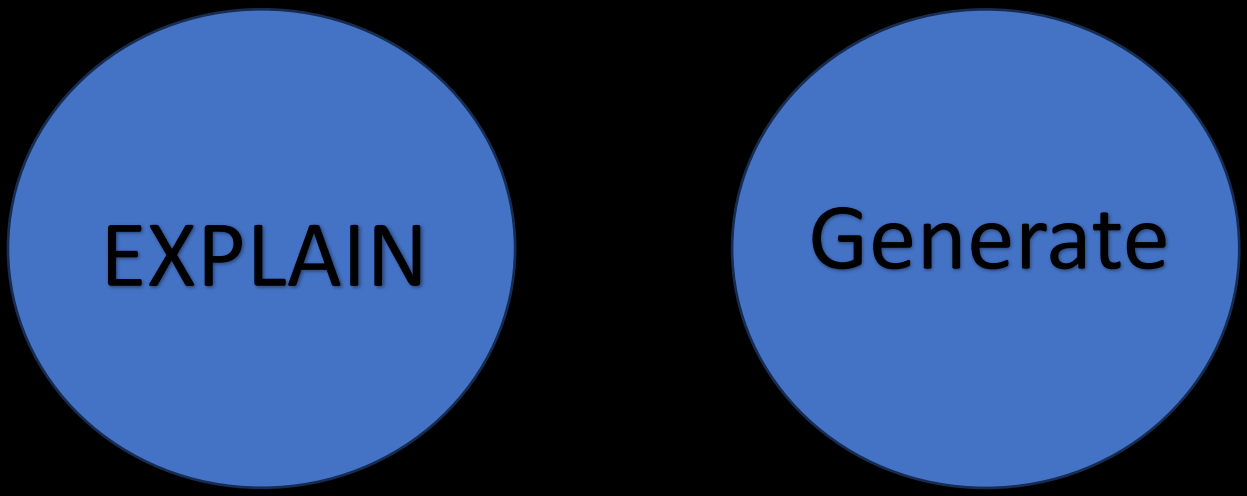
Examples:

- Write Python code.
- Generate a CNN model.
- Build a Flask API.
- Create a sorting algorithm.

The classifier output determines the execution path.

3. Intelligent Router

After classification, a routing component determines which pipeline should handle the request.



Route 1 — Code Explanation

If the user asks to explain code:

User

↓

Classifier

↓

Explain Code

↓

LLM

↓

Answer

Important

- **This path must NOT use RAG.**
- **No document retrieval.**
- **No Vector Database search.**
- **No embeddings.**
- **The LLM should directly analyze the provided code and generate an explanation.**

Route 2 — Code Generation

If the request is classified as Generate Code, the workflow becomes significantly more advanced.

User

↓

Classifier

↓

Generate Code

↓

RAG Retrieval

↓

Relevance Check

↓

LLM

↓

Generated Code

4. Retrieval-Augmented Generation (RAG)

The assistant should retrieve relevant programming knowledge from a **Chroma Vector Database** before generating code.

The retrieval pipeline includes:

- Document chunking
- Embedding generation
- Similarity search
- Top-k retrieval

5. Chroma Vector Database

Use Chroma as the vector store.

The database should contain:

- Documents
- Embeddings
- Metadata
- Source information

The retrieval module searches Chroma for documents most relevant to the user's programming request.

6. Retrieval Relevance Check

After retrieving documents, they must not be used immediately.

Instead, pass the retrieved context to another LLM acting as a Relevance Evaluator.

Its task is to answer:

"Is the retrieved context sufficiently relevant to answer the user's request?"

Possible outputs:

- Relevant
- Not Relevant

If Relevant

Proceed normally.

Retrieved Context

↓

Relevant

↓

LLM

↓

Generate Final Answer

If Not Relevant

Do **not** hallucinate.

Instead, respond to the user with a message such as:

"I couldn't find relevant knowledge for your request. Could you provide the correct solution or reference so I can learn it for future interactions?"

7. Human Feedback Learning

If the user provides a solution after retrieval fails:

Example:

User:

Here is the correct implementation...

The assistant should automatically:

1. Generate embeddings.
2. Create metadata.
3. Store the document.
4. Insert it into Chroma.

This enables the assistant to continuously expand its knowledge base over time.

Workflow:

User Solution

↓

Embedding Model

↓

Vector

↓

Chroma Database

8. Long-Term Conversation Memory

The chatbot should maintain conversational context using a memory module.

The memory stores information such as:

- Previous questions
- Previous generated code
- User preferences
- Programming language preference
- Framework preference
- Follow-up requests

The memory should be supplied to the LLM with each interaction so that conversations remain context-aware.

9. Code Generation

When code generation is requested, the LLM should produce:

- Complete source code
- Comments
- Best practices
- Error handling
- Explanation (optional)

The response should be formatted using Markdown code blocks.

10. Code Execution Tool

The assistant should integrate with an external **Code Runner Tool**.

The tool receives generated source code as a **string** and executes it.

The LLM itself should **not execute code**. Instead, it invokes the tool, which is responsible for:

- Executing the code in a controlled environment
- Capturing standard output
- Capturing standard error

- Returning execution results to the assistant

The assistant should then present the execution results to the user.